Assignment 5

Name: Trimaanpreet Kaur

UID: 22BCS13971

Section: 22BCS-IOT-605

Group: A

Easy

Write a Java program to calculate the sum of a list of integers using Autoboxing and Unboxing.

Implement methods to parse strings into their respective wrapper classes using Integer.parseInt()

Create an ArrayList of integers. Read a list of numbers from the user as a string. Convert the input into Integer objects using autoboxing. Compute the sum of the integers using unboxing

Display the result.

Sample Output

Enter numbers separated by spaces: 10 20 30 40 50

Sum of numbers: 150

Code

```
import java.util.ArrayList;
import java.util.Scanner;

public class SumWithAutoboxing {
   public static void main(String[] args) {

      Scanner scanner = new Scanner(System.in);

      // Step 1: Create an ArrayList of Integers
      ArrayList<Integer> numbers = new ArrayList<>();

      // Step 2: Read input from the user
      System.out.print("Enter numbers separated by spaces: ");
      String input = scanner.nextLine();

      // Step 3: Split the string and parse to Integer (Autoboxing)
      String[] parts = input.trim().split("\\s+");
      for (String part : parts) {
         int value = Integer.parseInt(part);  // Convert to primitive int
         numbers.add(value);                  // Autoboxing to Integer
      }

      // Step 4: Compute sum using unboxing
      int sum = 0;
      for (Integer number : numbers) {
```

```
            sum += number; // Unboxing occurs here
        }

        // Step 5: Display the result
        System.out.println("Sum of numbers: " + sum);
        scanner.close();
    }
}
```

Output

```
Enter numbers separated by spaces: 22 34 56 12 66 786
Sum of numbers: 976


...Program finished with exit code 0
Press ENTER to exit console.
```

Medium

Create a Student class and implement serialization & deserialization.

Define a Student class with id, name, and GPA. Implement Serializable interface. Create methods to: Serialize a Student object and save it to a file.

Deserialize the Student object from the file and display its details. Handle

FileNotFoundException, IOException, and ClassNotFoundException properly.

Student details saved successfully!

Reading from file...

Student ID: 101

Student Name: John Doe

Student GPA: 3.8.

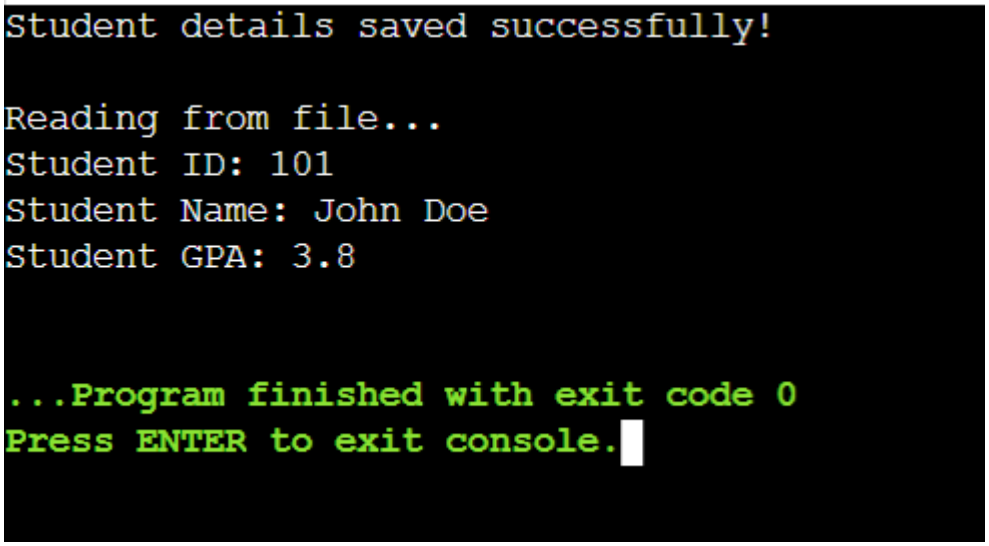
Code

```
import java.io.*;

// Step 1: Define Student class implementing Serializable
class Student implements Serializable {
    private int id;
    private String name;
    private double gpa;

    // Constructor
    public Student(int id, String name, double gpa) {
```

```
        this.id = id;
        this.name = name;
        this.gpa = gpa;
    }

    // Getters for displaying
    public int getId() { return id; }
    public String getName() { return name; }
    public double getGpa() { return gpa; }
}

public class StudentSerializationDemo {

    // Step 2: Serialize the Student object
    public static void serializeStudent(Student student, String filename) {
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream(filename))) {
            oos.writeObject(student);
            System.out.println("Student details saved successfully!");
        } catch (IOException e) {
            System.out.println("Error during serialization: " + e.getMessage());
        }
    }

    // Step 3: Deserialize the Student object
    public static Student deserializeStudent(String filename) {
        Student student = null;
        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream(filename))) {
            student = (Student) ois.readObject();
            System.out.println("\nReading from file...");
        } catch (FileNotFoundException e) {
            System.out.println("File not found: " + e.getMessage());
        } catch (IOException e) {
            System.out.println("Error during deserialization: " + e.getMessage());
        } catch (ClassNotFoundException e) {
            System.out.println("Class not found: " + e.getMessage());
        }
        return student;
    }

    public static void main(String[] args) {
        String filename = "student.ser";

        // Create a Student object
        Student student = new Student(101, "John Doe", 3.8);

        // Serialize the object
        serializeStudent(student, filename);

        // Deserialize the object
        Student deserializedStudent = deserializeStudent(filename);

        // Display student details
```

```
        if (deserializedStudent != null) {
            System.out.println("Student ID: " + deserializedStudent.getId());
            System.out.println("Student Name: " + deserializedStudent.getName());
            System.out.println("Student GPA: " + deserializedStudent.getGpa());
        }
    }
}
```

Output



Hard

Develop a menu-based application with the following options: 1. Add an Employee 2. Display All Employees 3. Exit

Create an Employee class with: Employee ID Name Designation Salary Implement file handling to store employee details persistently. Provide an interactive menu-based system for user input. Ensure proper exception handling.

Sample Output

Menu:

1. Add an Employee 2. Display All Employees 3. Exit…..

Choose an option:

1 Enter Employee ID: 1001

Enter Employee Name: Alice Enter Designation: Software Engineer Enter Salary: 60000 Employee added successfully!

Menu: 1. Add an Employee 2. Display All Employees 3. Exit

Choose an option:

2 Employee ID: 1001, Name: Alice, Designation: Software Engineer, Salary: 60000

Code

```
import java.io.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

// Step 1: Define Employee class
class Employee implements Serializable {
    private int id;
    private String name;
    private String designation;
    private double salary;

    public Employee(int id, String name, String designation, double salary) {
        this.id = id;
        this.name = name;
        this.designation = designation;
        this.salary = salary;
    }

    @Override
    public String toString() {
        return "Employee ID: " + id +
               ", Name: " + name +
               ", Designation: " + designation +
               ", Salary: " + salary;
    }
}

public class EmployeeManagementApp {

    private static final String FILE_NAME = "employees.dat";

    // Method to add an employee
    public static void addEmployee(Employee emp) {
        List<Employee> employees = loadEmployees();
        employees.add(emp);
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream(FILE_NAME))) {
            oos.writeObject(employees);
            System.out.println("Employee added successfully!");
        } catch (IOException e) {
            System.out.println("Error saving employee: " + e.getMessage());
        }
    }

    // Method to load all employees from file
    @SuppressWarnings("unchecked")
    public static List<Employee> loadEmployees() {
        List<Employee> employees = new ArrayList<>();
```

```
        File file = new File(FILE_NAME);
        if (!file.exists()) return employees;

        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(file)))
{
            employees = (List<Employee>) ois.readObject();
        } catch (IOException | ClassNotFoundException e) {
            System.out.println("Error loading employee data: " + e.getMessage());
        }
        return employees;
    }

    // Method to display all employees
    public static void displayAllEmployees() {
        List<Employee> employees = loadEmployees();
        if (employees.isEmpty()) {
            System.out.println("No employee records found.");
        } else {
            for (Employee emp : employees) {
                System.out.println(emp);
            }
        }
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int choice;

        do {
            System.out.println("\nMenu:");
            System.out.println("1. Add an Employee");
            System.out.println("2. Display All Employees");
            System.out.println("3. Exit");
            System.out.print("\nChoose an option: ");
            choice = scanner.nextInt();
            scanner.nextLine();  // Consume newline

            switch (choice) {
                case 1:
                    System.out.print("Enter Employee ID: ");
                    int id = scanner.nextInt();
                    scanner.nextLine();
                    System.out.print("Enter Employee Name: ");
                    String name = scanner.nextLine();
                    System.out.print("Enter Designation: ");
                    String designation = scanner.nextLine();
                    System.out.print("Enter Salary: ");
                    double salary = scanner.nextDouble();
                    Employee emp = new Employee(id, name, designation, salary);
                    addEmployee(emp);
                    break;
                case 2:
                    displayAllEmployees();
                    break;
```

```
                case 3:
                    System.out.println("Exiting program...");
                    break;
                default:
                    System.out.println("Invalid option! Try again.");
            }

        } while (choice != 3);

        scanner.close();
    }
}
```

Output

```
Menu:
1. Add an Employee
2. Display All Employees
3. Exit

Choose an option: 1
Enter Employee ID: 20
Enter Employee Name: Trimaan
Enter Designation: HR Manager
Enter Salary: 100000
Employee added successfully!

Menu:
1. Add an Employee
2. Display All Employees
3. Exit

Choose an option: 2
Employee ID: 20, Name: Trimaan, Designation: HR Manager, Salary: 100000.0

Menu:
1. Add an Employee
2. Display All Employees
3. Exit

Choose an option: 3
Exiting program...
```